```
#!/usr/bin/env python3
"""
HRV REAL (PhysioNet Fantasia, sujeito Y1,
jovem saudável) -> refractory_period
Fonte: Iyengar N, Peng C-K, Morin R,
Goldberger AL, Lipsitz LA. Age-related
alterations in the fractal scaling of
cardiac interbeat interval dynamics.
Am J Physiol 1996;271:1078-1084. Via
PhysioNet (Goldberger et al., 2000).
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d

K = 4
labels = ['Visão', 'Audição',
'Interocepção', 'Linguagem interna']
T = 400
kappa = 0.25
precision_ema = 0.05
ignition_threshold = 2.5
broadcast_gain_normal = 0.5
obs_noise = 0.3
refractory_period_default = 15
precision_ceiling_normal = 5.0

def true_signal(k, t):
    base_freqs = [0.02, 0.035, 0.015, 0.05]
    event_times = [100, 180, 260, 320]
```

```
    val = np.sin(2 * np.pi * base_freqs[k]
* t)
    if t > event_times[k]:
        val += 1.5
    return val

#
==========================================
=================
# 1. CARREGAR RR REAL E CALCULAR RMSSD REAL
(JANELA DESLIZANTE)
#
==========================================
=================
rr =
np.loadtxt('/home/claude/active_inference_w
orkspace/fantasia_Y1_RR_intervals.txt')  #
segundos
print(f"RR intervals carregados: {len(rr)}
batimentos reais (Fantasia Y1, PhysioNet)")
print(f"RR médio: {rr.mean()*1000:.1f} ms
(~{60/rr.mean():.0f} bpm) — plausível para
adulto jovem saudável")

def sliding_rmssd(rr_series, window=60,
step=30):
    """RMSSD real = sqrt(media((RR[i+1]-
RR[i])^2)) em janela deslizante de
batimentos."""
    diffs = np.diff(rr_series) * 1000  # ms
    vals = []
    for i in range(0, len(diffs) - window,
step):
        w = diffs[i:i+window]
```

```
        vals.append(np.sqrt(np.mean(w**2)))
    return np.array(vals)

rmssd_real = sliding_rmssd(rr)
print(f"RMSSD real: {len(rmssd_real)}
janelas, média {rmssd_real.mean():.1f} ms,
"
      f"min {rmssd_real.min():.1f} ms, max
{rmssd_real.max():.1f} ms")

# interpolar para T=400 pontos (para
alinhar com a simulação)
x_old = np.linspace(0, 1, len(rmssd_real))
x_new = np.linspace(0, 1, T)
hrv_real_series = interp1d(x_old,
rmssd_real, kind='linear')(x_new)

def hrv_to_refractory(hrv_series,
min_ref=8, max_ref=30):
    hrv_norm = (hrv_series -
hrv_series.min()) / (hrv_series.max() -
hrv_series.min() + 1e-9)
    return np.round(max_ref - (max_ref -
min_ref) * hrv_norm).astype(int)

refractory_from_real_hrv =
hrv_to_refractory(hrv_real_series)

#
==========================================
================
# 2. SIMULAÇÃO (mesma estrutura de antes)
#
==========================================
```

```
=================
def run_simulation(seed=7,
refractory_series=None,
refractory_fixed=refractory_period_default)
:
    np.random.seed(seed)
    mu = np.zeros(K)
    precision = np.ones(K) * 1.0
    err_ema_sq = np.ones(K) * 1.0
    refractory_until = np.zeros(K,
dtype=int)
    hist = {'ignition': np.zeros(T,
dtype=bool), 'abs_err': np.zeros((T, K)),
            'winner': np.full(T, -1)}

    for t in range(T):
        ref_period = refractory_series[t]
if refractory_series is not None else
refractory_fixed
        true_vals =
np.array([true_signal(k, t) for k in
range(K)])
        obs = true_vals +
np.random.randn(K) * obs_noise
        err = obs - mu
        salience = precision * err**2
        mu = mu + kappa * precision * err
        err_ema_sq = (1 - precision_ema) *
err_ema_sq + precision_ema * err**2
        precision = np.clip(1.0 / (0.1 +
err_ema_sq), 0.1, precision_ceiling_normal)
        eligible = np.array([salience[k] if
t >= refractory_until[k] else -1.0 for k in
range(K)])
```

```
        winner = int(np.argmax(eligible))
        ignite = eligible[winner] >
ignition_threshold
        if ignite:
            refractory_until[winner] = t +
ref_period
            for j in range(K):
                if j != winner:
                    mu[j] = mu[j] +
broadcast_gain_normal * (mu[winner] -
mu[j])
        hist['ignition'][t] = ignite
        hist['winner'][t] = winner if
ignite else -1
        hist['abs_err'][t] = np.abs(mu -
true_vals)
    return hist

hist_fixed = run_simulation(seed=7,
refractory_series=None)
hist_real_hrv = run_simulation(seed=7,
refractory_series=refractory_from_real_hrv)

print("\n" + "="*70)
print("REFRATÁRIO FIXO vs. REFRATÁRIO VIA
HRV REAL (Fantasia/PhysioNet)")
print("="*70)
print(f"Refratário fixo:
{refractory_period_default} passos
constantes")
print(f"Refratário via HRV real: varia
entre {refractory_from_real_hrv.min()} e
{refractory_from_real_hrv.max()} passos")
print(f"\nIgnições (fixo):
```

```
{hist_fixed['ignition'].sum()}")
print(f"Ignições (HRV real):
{hist_real_hrv['ignition'].sum()}")
print(f"Erro médio |mu-true| (fixo):
{hist_fixed['abs_err'].mean():.4f}")
print(f"Erro médio |mu-true| (HRV real):
{hist_real_hrv['abs_err'].mean():.4f}")

#
==========================================
================
# PLOTS
#
==========================================
================
fig, axes = plt.subplots(3, 1, figsize=(13,
10))

ax = axes[0]
ax.plot(rr[:2000]*1000, color='darkred',
linewidth=0.7)
ax.set_title('Dado real: intervalos R-R
(Fantasia Y1, PhysioNet) — primeiros 2000
batimentos')
ax.set_ylabel('RR (ms)')
ax.grid(True, alpha=0.3)

ax = axes[1]
ax.plot(hrv_real_series, color='crimson',
label='RMSSD real (Fantasia Y1),
interpolado p/ T=400')
ax.set_ylabel('RMSSD (ms)',
color='crimson')
ax2 = ax.twinx()
```

```
ax2.plot(refractory_from_real_hrv,
color='navy', alpha=0.6, label='período
refratário derivado')
ax2.set_ylabel('Refratário (passos)',
color='navy')
ax.set_title('RMSSD real -> período
refratário')
ax.grid(True, alpha=0.3)

ax = axes[2]
ign_fixed =
np.where(hist_fixed['ignition'])[0]
ign_real =
np.where(hist_real_hrv['ignition'])[0]
for t in ign_fixed:
    ax.scatter(t, 0, color='green', s=15,
alpha=0.6)
for t in ign_real:
    ax.scatter(t, 1, color='crimson', s=15,
alpha=0.6)
ax.set_yticks([0, 1]);
ax.set_yticklabels(['Refratário fixo',
'Refratário via HRV real'])
ax.set_xlabel('Tempo (passos)')
ax.set_title(f'Ignições: fixo (n=
{len(ign_fixed)}) vs. HRV real (n=
{len(ign_real)})')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('/home/claude/active_inference_
workspace/real_hrv_experiment.png',
dpi=150)
```

```
print("\nArquivo gerado:
real_hrv_experiment.png")
```